ZON TERRAS IN GENT

Terras Sun-Seeker: Zonoverzicht voor Gentse Terrassen

Webdevelopment Project - Voortgangsrapport

Wisdom Ononiba & Yoanna Oosterlinck

Universiteit Gent | Academiejaar 2025-2026 | 15 Maart 2026

Agenda

Overzicht van de presentatie

1. Introductie & Project Visie
2. Technische Architectuur
3. Wat We Hebben Bereikt
4. Technische Diepgang
5. Volgende Stappen
6. Afsluiting & Vragen

Project Visie & Doelstellingen

Wat is Zon Terras In Gent?

- Probleemstelling: In Gent is een zonnig terras vinden lastig. Kleine verschuivingen in zonpositie maken het verschil tussen zon of schaduw door kerktorens en gebouwen.
- Oplossing: Terras Sun-Seeker toont real-time welke Gentse cafeterrassen zonnig zijn via lichtintensiteit en kleurcodes (geel = zon, blauw = schaduw).
- Target doelgroep: Studenten (zonnig cafe tijdens pauze), remote workers (schaduwrijke terrassen bij hoge temperaturen), en fotografen (golden hour notificaties).
- Unique Value: Combinatie van real-time zonpositie, 3D-schaduwmodellering, weerdata en toeristische evenementen op een interactieve kaart.
- Use case 1: Student bij Korenmarkt zoekt zonnig cafe tijdens pauze.
- Use case 2: Remote worker zoekt schaduwrijk terras bij hittegolf.
- Use case 3: Fotograaf wil golden hour notificaties voor terrassen.

Project Scope

In scope vs. Out of scope

IN SCOPE

- REST API met content negotiation (JSON + HTML)
- Real-time zonpositie berekeningen (SunCalc3)
- Weerdata integratie (Open-Meteo API)
- Terras/Restaurant/Event data (Overpass + Stad Gent)
- CRUD operaties voor alle entiteiten
- Geospatiale queries (MongoDB 2dsphere)
- Hypermedia links (HATEOAS) in alle responses
    - HATEOAS: Hypermedia as the Engine of Application State
- Server-side Pug views voor HTML representaties
- Input validatie middleware
- Soft delete pattern met cascade

OUT OF SCOPE

- React frontend (gepland, nog niet gestart)
- API Data Integriteit Testing
- WebSocket/Socket.io real-time updates
- 3D shadow raycasting (Three.js)
- User authenticatie / autorisatie
- Leaflet kaart met:
    - schaduw-simulator
    - Heatmap

DEEL 2

Technische Architectuur

Tech Stack Overview

Volledige technologie stack

BACKEND

- Node.js + TypeScript 5.9
- Express 5.2.1
- Mongoose 9.2.1
- Pug 3.0.3 (templating)

DATABASE

- MongoDB
- 2dsphere indexes
- GeoJSON Points
- Polymorphic refs

EXTERNE API'S

- Open-Meteo (weer)
- Overpass API (OSM)
- Stad Gent API
- SunCalc3 (zon)

TOOLS

- node-cron (scheduling)
- express-validator
- nodemon (dev)
- ts-node

Polymorphe refs: SunData verwijst dynamisch naar Terras, Restaurant of Event via locationType`
express-validator: valideert user input in route-handlers (coordinaten, MongoDB IDs, datums, enums)

Backend Architectuur

Express 5 + TypeScript + MVC patroon

API DESIGN (REST)

- 6 route groepen: /api/terrasen, /api/restaurants, /api/events, /api/sun, /api/weather, /api/search
- Volledige CRUD: GET, POST, PUT, PATCH, DELETE
- Content negotiation: JSON + HTML via Accept header
- HATEOAS: hypermedia links in alle responses
- Stateless (REST): geen sessies, elk verzoek onafhankelijk
- Soft delete: isDeleted flag + deletedAt timestamp
- Cascade delete: SunData verwijderd bij entiteit delete

MVC STRUCTUUR

- Models: Mongoose schemas met validatie en indexes
- Views: Pug templates gegroepeerd per entiteit
- Controllers: Factory pattern (baseController.ts)
- Routes: Aparte bestanden per resource type
- Middleware: Input validatie (express-validator)
- Services: Externe API integraties + data fetchers
- Cron: Geautomatiseerde data sync en cache updates

Database Design

MongoDB met Mongoose ODM - 5 Collections

Collection	Velden	Indexes
Terras	name, address, identifier (OSM), location (GeoJSON), intensity (0-100), isDeleted	2dsphere, intensity
Restaurant	name, address, cuisine, rating (0-5), phone, website, location, intensity	2dsphere, rating+intensity, name+cuisine
Event	title, address, date_start, date_end, description, url, location, intensity	2dsphere, date_start+date_end
Weather	timestamp, temperature, cloudCover, cloudFactor, uvIndex, windspeed, location	timestamp, 2dsphere
SunData	locationRef (locationRef (ObjectId -> Terras/Restaurant/Event)), locationType, dateTime, intensity, azimuth, altitude, goldenHour	unique(ref+type+time)

Kenmerken: GeoJSON Points voor locatie | 2dsphere indexes | Polymorphe referenties | Soft delete pattern

Infrastructure & DevOps

Deployment configuratie en server setup

HUIDIGE SETUP

- Ubuntu server via UGent
- Node.js runtime + MongoDB lokaal
- NGINX
- SSL certificaat via Let's Encrypt (HTTPS)
- HTTP -> HTTPS redirect geconfigureerd
- Statische assets caching (1 uur) via NGINX
- Environment variabelen via .env.example bestand

GEPLANDE VERBETERINGEN

- Automated testing voor deployment
- Structured logging
- Database backups (mongodump)
- Monitoring & alerting

DEEL 3

Wat We Hebben Bereikt

Achievements Overview

Projectvoortgang in cijfers

50

Issues Gesloten

van 84 totaal (58%)

128

Commits

actieve development

2

Contributors

Wisdom & Yoanna

33

API Endpoints

REST endpoints

Voortgang: 50 / 86 issues (58%)

58%

Backend Achievements - Features

Alle geimplementeerde backend features met issue referenties

- Volledige CRUD voor Terrassen (#87, #47, #46, #9, #8) - GET/POST/PUT/PATCH/DELETE
- Volledige CRUD voor Restaurants (#88, #50, #49, #48, #40) - inclusief rating systeem
- Volledige CRUD voor Events (#89, #56, #55, #54, #39) - met datum filtering
- Sun Position Endpoints (#16, #59, #58, #57, #37) - real-time zonpositie berekeningen
- Weather Integration (#17, #53, #52, #51, #38) - Open-Meteo API integratie
- Aggregated Data Endpoints (#18) - gecombineerde queries over meerdere entiteiten
- Search Controller (#32) - full-text search, intensity filters, geospatiale radius zoekopdrachten
- Fetch Terras Data (#100) - automatische sync via Overpass API
- Fetch Restaurant Data (#14) - Overpass API restaurant sync
- Fetch Tourist Events (#13) - Stad Gent API event sync
- Fetch Weather Data (#12) - Open-Meteo cron job

Backend Achievements - Technisch

Architecturele en technische implementaties

- Database indexing voor alle modellen (#41-#45) - 2dsphere, compound, unique indexes
- Factory Pattern: baseController.ts genereert CRUD handlers, elimineert code duplicatie
- Content Negotiation (#90) - automatische JSON/HTML response op basis van Accept header
- Hypermedia Links (#91, #97) - zowel JSON als HTML responses
- Input Validation Middleware (#92) - express-validator voor coords, IDs, datums, enums
- Weather Cron Job (#81) - geautomatiseerde 15-min weather cache met grid-based optimalisatie
- Soft Delete Pattern - isDeleted flag + deletedAt timestamp + cascade naar SunData
- Polymorphe Referenties - SunData koppelt aan Terras/Restaurant/Event via locationType
- Overpass API Wrapper - retry logic (3 attempts, 30s backoff) voor 429/504 errors
- Server Update Script (#34) - geautomatiseerd deployment na git pull
- NGINX Configuration (#33) - reverse proxy, SSL, caching

Frontend Achievements - Pug Views

Server-side rendered HTML views (#80)

PAGINA'S

- Homepage: API welkomstpagina met navigatie
- Terras Lijst: overzicht met zonscore en adres
- Terras Detail: individuele terras informatie
- Restaurant Lijst: overzicht met ratings
- Restaurant Detail: cuisine, contact, openingsuren
- Event Lijst: huidige en toekomstige evenementen
- Event Detail: datum range, locatie, beschrijving
- Sun Data Display: positie, altitude, golden hours
- Sun Batch: bulk zonberekeningen
- Error pagina: gebruikersvriendelijke foutmeldingen

DEEL 4

Technische Diepgang

Interessante Technische Uitdagingen

Complexe problemen en hun oplossingen

1. Geospatiale Queries & Performance

Probleem: Zoeken naar terrassen/restaurants binnen een radius vereist complexe geospatiale berekeningen op grote datasets.

Oplossing: MongoDB 2dsphere indexes + $geoNear aggregation pipeline + grid-based coordinate grouping (3 decimalen ~ 111m) om weather API calls te batcheren.

2. Real-time Zonintensiteit Berekening

Probleem: Combinatie van astronomische zonpositie en bewolking tot een bruikbare intensiteitswaarde (0-100).

Oplossing: SunCalc3 voor altitude/azimuth + Open-Meteo cloudCover * 0.8 factor + hourly caching in SunData collectie met polymorphe referenties.

3. Content Negotiation met Hypermedia

Probleem: Dezelfde endpoints moeten zowel bruikbare HTML (met navigatie) als machine-readable JSON serveren.

Oplossing: Accept header parsing in controllers + factory pattern dat automatisch links genereert + Pug templates voor HTML + JSON links array.

Interessante Technische Uitdagingen

Complexe problemen en hun oplossingen

1. Geospatiale Queries & Performance

Probleem: Gebruikers willen terrassen/restaurants vinden binnen een bepaalde radius (bijv. 500m). MongoDB moet voor elke query de afstand berekenen tussen de gebruiker en honderden locaties met coordinaten.

Oplossing:

MongoDB 2dsphere Index
- 2dsphere index op location veld (GeoJSON Point) in elke collectie

$geoNear Aggregation Pipeline
- Eerste stage: filtert én sorteert op afstand in één enkele query

Grid-based Coordinate Grouping (weatherCron.ts)
- Lat/lng afronden op 3 decimalen (~111m nauwkeurigheid)
- Locaties in hetzelfde grid-vakje delen dezelfde weather API call

Interessante Technische Uitdagingen

Complexe problemen en hun oplossingen

2. Real-time Zonintensiteit Berekening

Probleem:
De intensiteitswaarde (0-100) per locatie moet twee bronnen combineren: de astronomische zonpositie (hoek van de zon boven de horizon) en de actuele bewolking. Beide veranderen continu.

Oplossing:
SunCalc3 library: Berekent altitude (hoek boven horizon, in radialen) en azimuth (kompasrichting) op basis van lat/lng/tijd

Basis-intensiteit
- `basisIntensiteit = altitude > 0 ? Math.sin(altitude) * 100 : 0`
- Sinus van de hoogte geeft realistische lichtsterkte (lager bij lage zon, maximum bij 90°)

Cloud factor correctie
- Open-Meteo levert `cloudCover` (0–100%)
- Omgerekend naar `cloudFactor = cloudCover * 0.8`
- Reduceert de intensiteit proportioneel (bv 50% bewolking = 40% reductive)

Eindformule: `intensity = basisIntensiteit * (1 - cloudFactor / 100)`

Golden hour & caching
- Golden hour tijden (`dawnStart`, `dawnEnd`, `duskStart`, `duskEnd`) worden opgeslagen voor fotografen
- Resultaat per uur gecached in `SunData` collectie met unieke compound index op `(locationRef + locationType + dateTime)` zodat dezelfde berekening niet wordt herhaald

Interessante Technische Uitdagingen

Complexe problemen en hun oplossingen

3. Content Negotiation met Hypermedia

Probleem:
Volgens de REST architecturale stijl moet dezelfde URL zowel HTML voor browsers als JSON voor machines serveren. Beide representaties moeten navigeerbare links bevatten (HATEOAS), maar in een ander formaat.

Oplossing:

Format detectie
- Controller checkt `req.accepts(['json', 'html'])` om gewenst formaat te bepalen

JSON Response
- Bevat `links` array met `{ rel, href, method }` objecten
- Voorbeeld: `{ rel: "self", href: "/api/terrasen/abc123", method: "GET" }`
- `{ rel: "collection", href: "/api/terrasen", method: "GET" }`

HTML Response
- Pug template rendert dezelfde data met `<a href="...">` tags
- Inclusief navigatie-elementen

baseController factory
- Genereert automatisch juiste links per entiteit type
- Nieuwe entiteiten krijgen direct correcte hypermedia controls

Validatie errors
- Content-negotiated responses:
- JSON: object met `errors` array
- HTML: error pagina

Code Quality & Best Practices

Design patterns en architecturale keuzes

DESIGN PATTERNS

- Factory Pattern: baseController genereert CRUD handlers automatisch - voorkomt code duplicatie
- MVC Architecture: strikte scheiding Model-View-Controller met aparte routes
- Soft Delete Pattern: isDeleted + deletedAt vlag voor data behoud
- Polymorphe Referenties: SunData koppelt aan elk entity type via discriminator
- Service Layer: externe API calls afgezonderd in services/
- Middleware Pattern: validatie als herbruikbare middleware chains

BEST PRACTICES

- TypeScript: type-safe code met interfaces en strict mode
- Environment Variables: gevoelige data in .env (niet in code)
- Error Handling: try-catch + content-negotiated error responses
- REST Principes: correct HTTP methode gebruik (GET/POST/PUT/PATCH/DELETE)
- Git Workflow: feature branches + merge via pull requests

Security Implementaties

Beveiligingsmaatregelen in de applicatie

- Input Validatie: express-validator voor alle user input (coordinaten, IDs, datums, enums)
- MongoDB ObjectId Validatie: voorkomt NoSQL injection via ongeldig ID formaat
- Environment Variables: database URI, API URLs en PORT niet hardcoded in broncode
- NGINX Reverse Proxy: Node.js niet direct blootgesteld aan publiek internet
- SSL/TLS: Let's Encrypt certificaat voor HTTPS encryptie
- HTTP -> HTTPS Redirect: alle verkeer geforceerd over encryptie
- Hostname Validatie: NGINX geeft 404 voor onbekende hostnames
- Soft Delete: data wordt niet fysiek verwijderd, voorkomt accidenteel dataverlies
- Stateless API: geen sessie-opslag, conformeert aan REST principe

Performance & Optimalisatie

Strategieen voor snelle en efficiente data verwerking

- Database Indexing: 2dsphere indexes voor geospatiale queries, compound indexes voor sortering
- Grid-based Weather Batching: rondt coordinaten af op 3 decimalen (~111m) om API calls te reduceren
- Weather Cache: 15-minuten TTL voorkomt herhaalde externe API calls
- SunData Hourly Cache: zonberekeningen per uur gecached, niet per request
- Compound Unique Indexes: SunData (locationRef + locationType + dateTime) voorkomt dubbele entries
- Efficient Soft Delete: isDeleted filter in alle GET queries via Mongoose pre-hooks
- NGINX Static Caching: assets gecached voor 1 uur, vermindert server belasting
- Batch Sun Endpoint: meerdere locaties in een enkele API call verwerkt
- Node-cron Scheduling: achtergrond jobs voor data sync zonder API request overhead

DEEL 5

Volgende Stappen

Open Issues Analyse

31 open issues - gecategoriseerd naar prioriteit

Categorie	Prio
Testing (12 issues)	HIGH
React Frontend (10 issues)	HIGH
Data Seeding Scripts (3 issues)	MED
WebSocket/Real-time (2 issues)	MED
3D Shadow Rendering (2 issues)	LOW

Volgende Targets - Must Have

Prioriteit 1 features voor de volgende fase

- #66 Setup for tests - Jest/Vitest configuratie, test database, test utilities
- #67-#70 Model Tests - Schema validatie, indexes en constraints voor Terras, Restaurant, Event, SunData
- #71 GeoSpatial Query Tests - verificatie van 2dsphere queries en $geoNear pipelines
- #72-#76 Controller Logic Tests - unit tests voor alle controller functies per entiteit
- #77 Connection & Error Handling Tests - database connectie en error flow validatie
- #82 React Frontend Setup - Vite + TypeScript + React + Leaflet + TanStack Query
- #23 Leaflet Map - interactieve kaart met terras/restaurant/event markers
- #84-#86 Data Seeding Scripts - CLI scripts voor terras, event, restaurant data import

Volgende Targets - Nice to Have

Prioriteit 2 features en toekomstige verbeteringen

- #93, #94 WebSocket/Socket.io - real-time sun en weather updates naar frontend
- #24 Shadow Rendering - leaflet-shadow-simulator integratie voor visuele schaduwweergave
- #25 Future-Sun-Slider - SVG cirkeldiagram met uurlijkse zonintensiteit voorspelling
- #26 Search & Filters - frontend zoekfunctie op naam, zonscore, weercondities
- #27 Asynchrone Communicatie - TanStack Query + Socket.io in React
- #28 Weather Widgets - floating widget met UV-index, wind, temperatuur
- #29 Legend & Info - kaartlegende met kleurcodes en filteropties
- #30 Client-side Optimalisaties - caching, lazy loading, skeleton screens
- #31 Progressive Rendering - markers en heatmaps incrementeel laden
- #83 3D Shadow Raycasting - Three.js server-side shadow berekeningen
- #11 Fetch Gent 3D CSV - 3D gebouwdata voor schaduwmodellering

DEEL 6

Afsluiting

Lessons Learned

Reflectie op het development proces

WAT GING GOED

- REST architectuur: content negotiation en HATEOAS correct geimplementeerd
- Factory Pattern: code duplicatie sterk verminderd via baseController
- MongoDB geospatiale features: krachtig voor locatie-gebaseerde queries
- TypeScript: type safety heeft bugs in vroeg stadium voorkomen
- Externe API integratie: Open-Meteo + Overpass + Stad Gent soepel verlopen
- Git workflow: 128 commits, feature branches, pull requests
- Issue tracking: duidelijke planning via GitHub Issues

WAT KAN BETER

- Testing eerder starten: tests pas in fase 2 is een risico
- Documentatie: API docs formaliseren (Swagger/OpenAPI)
- Error handling: meer gestructureerde error responses
- Monitoring: geen health checks of logging framework
- Database migrations: schema versioning ontbreekt

Requirements Mapping

WEBDEV.html vereisten vs. onze implementatie

Vereiste (WEBDEV.html)	Status	Implementatie Details
Gegevensmodellering	DONE	5 Mongoose schemas, validatie, indexes, relaties
Gegevensvalidatie	DONE	express-validator: coords, IDs, datums, enums (#92)
HTML representaties	DONE	Pug templates per entiteit, layout, navigatie (#80)
JSON representaties + links	DONE	HATEOAS links in alle JSON responses (#91, #97)
Content negotiation	DONE	Accept header -> JSON of HTML (#90)
REST principes	DONE	GET/POST/PUT/PATCH/DELETE, stateless, hypermedia
Server deployment + NGINX	DONE	Ubuntu + NGINX + SSL + auto-start (#33, #34)
Soft delete (audioboeken)	DONE	isDeleted + deletedAt + cascade cleanup
min. 20 commits	DONE	128 commits over meerdere branches

Vragen?

Bedankt voor jullie aandacht!

Wisdom Ononiba & Yoanna Oosterlinck

Universiteit Gent | Webdevelopment | Maart 2026

github.ugent.be/wononiba/Zon-Terras-In-Gent